

Backend Architecture, Completed Work & Integration Guide

Author: Member 1 (Backend & Database Architect) | SIH Problem Statement 34 | Version: 1.0.0

1. Executive Summary of Completed Work

Member 1 has built, compiled, verified, and committed the entire NestJS API Gateway, Multi-Database Layer, and Business Logic Engines into the backend/ codebase. All 13 sub-part commits have been pushed to GitHub.

Component	Technology	Status / Access
API Gateway & Backend	NestJS + TypeScript	Live at http://localhost:3000/api/docs
Relational Database	PostgreSQL + Prisma ORM	12 Models defined (Users, Notices, Payments, etc.)
Graph Database	Neo4j	Retailer → Distributor → Manufacturer Traceability
Object Storage	MinIO / AWS S3 SDK	Presigned URLs for Images, Evidence & PDFs
Auth & RBAC	JWT + Keycloak Guards	4 Roles (Citizen, Business, Inspector, Controller)
Real-Time WebSockets	Socket.IO Gateway	Live status push updates for complaints

2. Core Business Logic Engines Implemented

- **Case Workflow State Machine:** Enforces transitions RECEIVED → ASSIGNED → INSPECTED → NOTICE_ISSUED → COMPOUNDED/PROSECUTION → RESOLVED with *inspector-configurable custom deadline days/dates*.
- **Repeat Offender Calculator:** Normalizes product and manufacturer names to automatically assign 1st, 2nd, or 3rd offence statutory fine bounds.
- **Dynamic Notice Renderer:** Substitutes 24 legal placeholders across 4 notice types (Improvement, Seizure, Panchanama, Compounded) in English & Marathi.
- **Anti-Corruption Audit Logger:** Creates an append-only, tamper-evident SHA-256 hash log for every status change and officer action.
- **Payments Service:** Integrates Razorpay with duplicate webhook event protection using razorpay_event_id.

3. Independent REST Endpoints (Available Right Now for Frontends)

These endpoints are fully functional and do not depend on external AI services. Frontend developers (Member 3 & Member 4) can build against them immediately:

Endpoint / Method	Target Role	Description / Usage
-------------------	-------------	---------------------

POST /api/v1/auth/login POST /api/v1/auth/register	All Users	User signup & JWT authentication
GET /api/v1/knowledge-base/rulebook GET /penalties	All Users	Query statutory rulebook, penalty matrix, and exemptions
POST /api/v1/complaints GET /complaints/:id/status	Citizen / Inspector	File complaint & get step-by-step status tracker
POST /api/v1/businesses GET /businesses	Business / Inspector	Register business & autocomplete business search
GET /api/v1/offences/history	Inspector	Query offence history by product & manufacturer
GET /api/v1/controller/dashboard/stats	Controller	Statewide analytics, 1st vs 2nd offence counts, case load
POST /api/v1/controller/compoundi ng/:id/action	Controller	Approve, Reject (+comments), or Escalate to Prosecution
POST /api/v1/controller/supply-cha in/multi-tier GET /supply-chain/trace/:gstin	Controller	Link & trace Retailer → Distributor → Manufacturer graph in Neo4j
POST /api/v1/payments/initiate	Business	Create Razorpay payment order for notice fines

4. Dependent Endpoints (Interacts with Member 2 AI/OCR & Razorpay)

These endpoints interact with Member 2's Python FastAPI AI service or Razorpay payment webhooks:

Endpoint / Method	Dependent On	Description & Contract
POST /api/v1/inspections/:id/ocr-result	Member 2 (FastAPI /ocr/scan)	Backend receives packaging scans, calls FastAPI OCR, saves JSON in DB, returns editable fields to Inspector App.
POST /api/v1/businesses/self-check	Member 2 (FastAPI /ocr/self-check)	Private self-check. Backend proxies images to FastAPI, saves report in private table isolated from inspectors.
POST /api/v1/notices	Member 2 (FastAPI /notices/generate)	Calls FastAPI NLP notice text generator, populates 24 placeholders across EN/MR templates, exports PDF to MinIO.
POST /api/v1/payments/webhook	Member 6 & Razorpay	Razorpay payment webhook callback. Deduplicates using razorpay_event_id, updates payment to PAID, resolves case.

5. Member-by-Member Integration Guide

Member 2 (AI/OCR Engineer): Expose FastAPI endpoints at /ocr/scan, /ocr/self-check, and /notices/generate adhering to JSON schemas in RULE_ENGINE_CONTRACT.md.

Member 3 (Flutter Mobile App): Connect Inspector & Business screens to <http://localhost:3000/api/v1/>. Refer to Swagger UI at <http://localhost:3000/api/docs>.

Member 4 (Next.js Web Developer): Connect Citizen Complaint & Controller Dashboard screens to backend REST APIs. Render Neo4j graph using GET /controller/supply-chain/trace/:gstin.

Member 6 (Integrations & QA Owner): Use Razorpay test keys with POST /payments/initiate and test webhook callbacks with POST /payments/webhook.